

Abstract

Motivation

1. Safety-Critical Systems: DNNs requires rigorous testing and assurances in applications that can affect human life and property
2. Scalability Challenge: Verification of DNNs suffers from the curse of dimensionality

Proposed Solution

1. By combining falsification and verification, VeRAPaK can efficiently attempt to refute the robustness property prior to engaging in expensive verification
2. VeRAPaK uses partitioning to divide the input space, leading to efficient processing and increased adversarial diversity

Key Contributions

- **Modular Framework:** Swappable verification, falsification, and partitioning strategies for increased versatility
- **Randomized Fast Gradient Sign Method (R-FGSM):** Improved adversarial generation for increased diversity over traditional FGSM while maintaining efficiency.
- **Diverse Adversarial Dataset Generation:** Partitioning and R-FGSM enable a diverse set of examples, which can be utilized for network hardness tuning.

Verification Engine

- VeRAPaK currently integrates the ETH robustness analyzer (ERAN) which uses abstract transformers to over-approximate the network's output region, yielding a hyper-box that can be checked for safety properties [1]
- The addition of a falsification engine can remove spurious counterexamples generated by the over-approximation, enhancing the resolution of claims being made

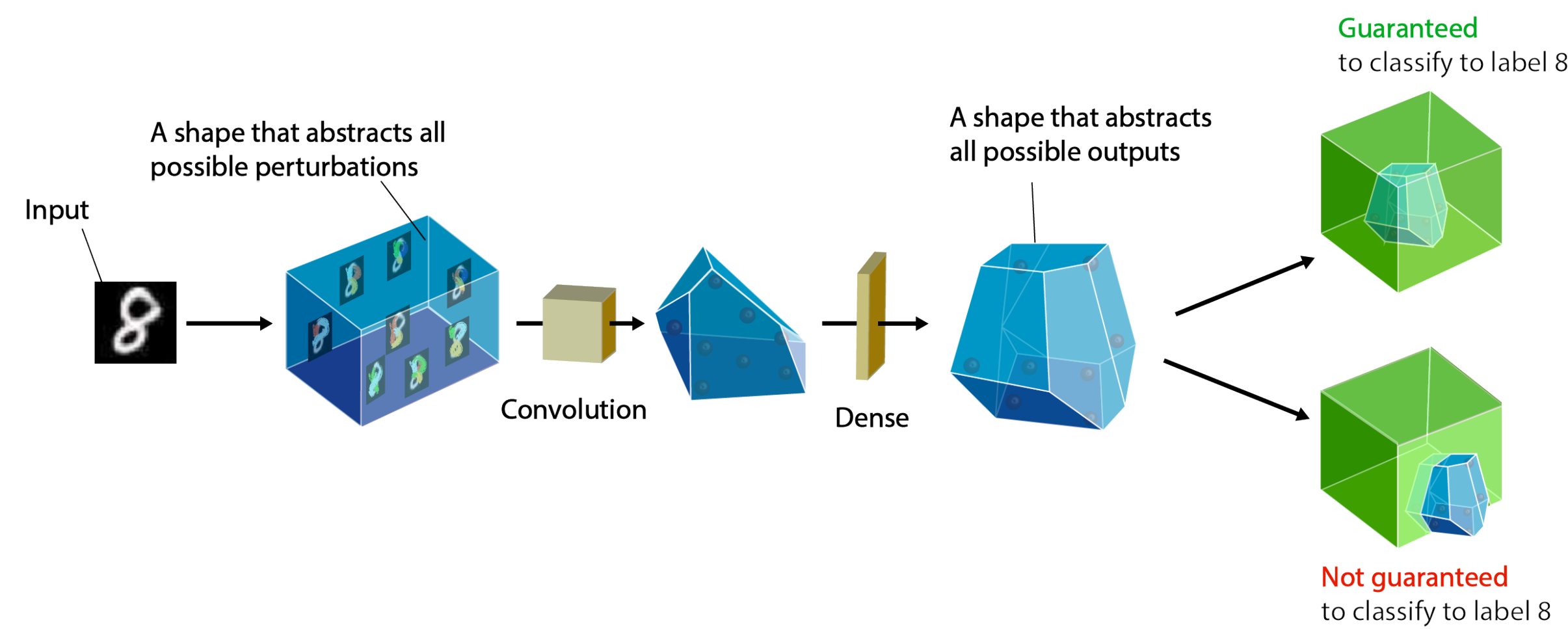


Figure 1. Over approximation with abstract interpretation using Eth Robustness Analyzer [1]

Partitioning Engine

- Supports different partitioning strategies, currently implementing Largest Dimension First.

Largest First Partitioning

1. Given an initial region: low = [0,0], high = [8,4], Dimension 0 is the largest and splits first
2. Size increment is selected as $(High_0 - Low_0)/numSplits \rightarrow (8 - 0)/2 = 4$
3. calculate sub regions for number of splits
 - sub-region 0: low = [0,0] and high = [4, 4]
 - sub-region 1: low = [4,0] and high = [8,4]
4. Split on second dimension with size increment: $(High_1 - Low_1)/numSplits \rightarrow (4 - 0)/2 = 2$
 - split sub-region 0: sub-region 3: low = [0,0], high = [4,2] and sub-region 4: low = [0,2], high = [4,4]
 - split sub-region 1: sub-region 5: low = [4,0], high = [8,2] and sub-region 6: low = [4,2], high = [8,4]

The VeRAPaK Framework

- Input region is defined as a point with radius, a point with per-dimension radius, or upper/lower bounds
- Iteratively processes subregions from the UNKNOWN and SOME_UNSAFE sets
- The Falsification engine (Falsify(r)) efficiently refutes the safety property, eliminating unsafe regions and providing counterexamples for placement into SOME_UNSAFE

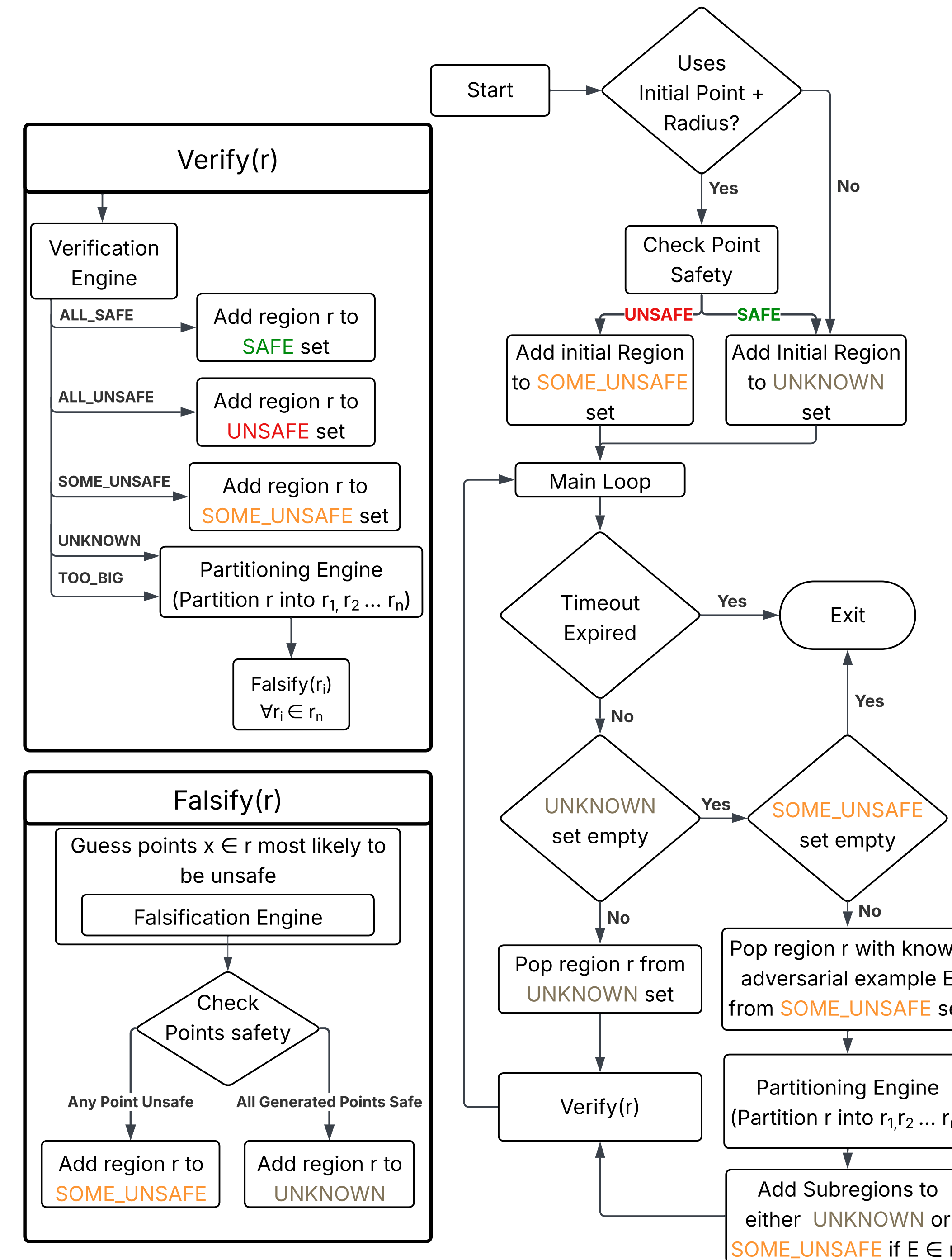


Figure 2. The VeRAPaK main algorithm

Randomized Fast Gradient Sign Method (R-FGSM)

The R-FGSM equation can be seen below:

$$x_{adv} = x + \epsilon \times [sign(\Delta_x J(\theta, x, y)) \times M + R \times (1 - M)] \quad (1)$$

where x is a point in the input space, M is a binary mask that determines if a dimension is updated by FGSM or by a random value R

Dimension Ranking Heuristics

Dimension Ranking Heuristics set the mask, M , to prioritize high-contribution dimensions for FGSM while randomizing others, achieving increased diversity and efficiency

1. Random - Random selection of dimensions
2. IntelliFeature - Uses knowledge of classification regions in training set
3. Gradient-based - Uses magnitude of the cost function gradient

Initial HiggsML Uncertainty Results

- Goal: Verify and harden a DNN against systematic biases
- μ : The signal strength of the Higgs boson particle within a given set
- Challenge: Real-world systematic uncertainties are unknown, biasing the model's estimator, as models are trained under specific assumptions of these systematic errors
- Method: use VeRAPaK to generate adversarial dataset and harden network
- Network: Simple FFNN with node and layer counts [28, 128, 256, 128, 1]

Model	MAE	RMSE	Avg. μ prediction (Target $\mu = 1$)
Base	17.178	126.335	4.551
Tuned	9.078	34.0327	0.817

Table 1. Hardness tuning results on test set

- Tuning with VeRAPaK's adversarial dataset significantly reduced μ prediction error
- Size of dataset constrained size of network and dataset generation time

Ongoing Work

Termination Proof

1. Proving termination is a non-trivial challenge because of the continuous nature of the input regions and their iterative subdivision within the framework
2. Use of NumPY within VeRAPaK complicates verification of Python code base
3. Working on modeling key aspects of VeRAPaK within Dafny to assist in proving properties of our tool.
4. Leveraging Fuzzing to further analyze the performance and robustness of VeRAPaK

Benchmark and Case Studies

1. Comprehensive benchmarking of VeRAPaK in this form on the VNN-COMP datasets
2. Refining initial findings on the HiggsML Uncertainty Challenge using VeRAPaK to harden DNNs against systematic variations [2]
3. Use VeRAPaK to verify and harden wind turbine fault classification networks

Code and Tool Availability

VeRAPaK is open source and available for use

1. Our github repository can be found here: <https://github.com/formal-verification-research/VERAPAK>
2. Documentation and Install steps can be found at: www.verapak.net

References

- [1] G. Singh, T. Gehr, M. Püschel, and M. Vechev, "An abstract domain for certifying neural networks," *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, pp. 1–30, Jan. 2019. [Online]. Available: <https://dl.acm.org/doi/10.1145/3290354>
- [2] W. Bhimji, P. Calafiura, R. Chakkappai, P.-W. Chang, Y.-T. Chou, S. Diefenbacher, J. Dudley, S. Farrell, A. Ghosh, I. Guyon, C. Harris, S.-C. Hsu, E. E. Khoda, R. Lyscar, A. Michon, B. Nachman, P. Nugent, M. Reymond, D. Rousseau, B. Sluijter, B. Thorne, I. Ullah, and Y. Zhang, "FAIR Universe HiggsML Uncertainty Challenge Competition," Sep. 2025, arXiv:2410.02867 [hep-ph]. [Online]. Available: <http://arxiv.org/abs/2410.02867>

Acknowledgments

- This work was supported by donations from Adobe Systems Inc.
- Special thanks to Dr. Shih-Chieh Hsu of University of Washington for the introduction and guidance on the Higgs ML challenge